

Meeting 1. The Neuron and the Gate

Act 1 The Biological Neuron · Act 2 Logic Gates and Certainty · Act 3 Computing a Neuron, Approximately

About these notes. There is no textbook to buy in CS 301: these notes are the course’s text, written by the instructors and posted on Canvas as each meeting is reached. They follow the meetings themselves — including questions and examples that came up in class — and, together with the assignment notebook you build, they are what the exams are set from. Logistics, grading, and the schedule live in the syllabus.

Unit I is organized around a single question:

*A logic gate is **written**. Could a neuron be **taught**?*

This first meeting introduces the two objects in that sentence — the biological neuron, a living cell, and the logic gate, the exact device computers are made of — and ends with a 1943 idea that connects them: a neuron, written as arithmetic.

Act 1. The Biological Neuron

1.1 Neurons and cell types

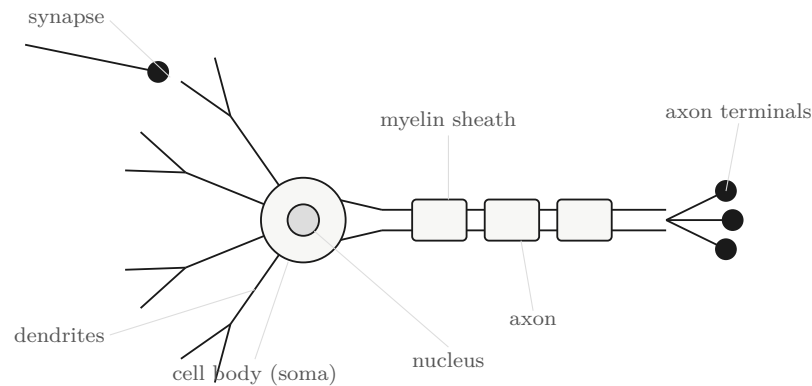
Your brain runs on roughly **86 billion** neurons. For decades the standard figure was “100 billion” — a round number that no measurement established. A laboratory technique gave a measured estimate: dissolve brain tissue into a suspension, count the cell nuclei in samples of it, and scale up. The answer comes out near 86 billion on average — about 16 billion in the cerebral cortex and about 69 billion in the cerebellum, with variation from person to person. The lesson for working with data: ask how a number was measured, rather than accepting it because it is often repeated.

A neuron is a cell — a strange-looking one, as the next section shows — and the 86 billion are not one cell repeated. Neurons come in **kinds**, differing in shape, chemistry, connections, and response. A recent atlas, built by reading which genes individual brain cells express, groups them into 461 clusters and 3,313 finer subclusters. Those counts include both neurons and non-neuronal cells.

Many cells share a type, and small circuits with similar arrangements also recur across the brain. This suggests a useful comparison with computers: complex systems can be built from repeated parts. Artificial neural networks also reuse small modules at a much larger scale.

The counts: Azevedo et al., *J. Comp. Neurol.* 513 (2009), 532–541, doi:10.1002/cne.21974. The cell-type atlas: *Transcriptomic diversity of cell types across the adult human brain*, *Science* 382 (2023), doi:10.1126/science.add7046.

1.2 What one cell does



The branched structures on the left are **dendrites**, from the Greek *dendron*, a tree — and they do look like the branches of one. Signals from other neurons, chemical or electrical, arrive there, at contact points called **synapses**. What arrives is gathered into the **cell body** (the soma), and when the cell responds, the response travels down the **axon** — the long fiber to the right — and out through the axon terminals to other cells.

The operation of the cell, reduced to four steps:

1. Signals **arrive** at the dendrites, at the synapses.
2. Each synapse has a **strength**. One way to say it: a synapse decides how much attention its input gets. A strong synapse passes its signal on with force; a weak one barely registers it; some push in the negative direction and *suppress* the cell instead.
3. The cell body **adds up** what arrives — each signal counted in proportion to the strength of the synapse it crossed.
4. Past a **threshold**, the cell **fires**: a spike travels down the axon to the next cells. Below the threshold it stays quiet. The threshold is the cell's own; different neurons have different ones.

This is a simplified description; different kinds of neuron behave differently. Keep the four steps — arrive, weigh, add, compare — in mind. In Act 3 we will express them as arithmetic.

1.3 Nothing here is exact

Nothing in the previous section behaves like a symbol in an equation.

- **Membrane voltage is continuous.** The voltage across the cell's membrane is not restricted to two levels; it moves through every value in a range. (A range, not an unbounded scale — the cell regulates itself, and activity that ran away unchecked would destroy it.)
- **Signals are pulses, not values.** A neuron does not hand the next cell a number. It emits spikes, and what carries information is their *rate* and their *timing*.
- **The contact is chemical.** At most synapses the signal crosses the gap as a puff of neurotransmitter molecules — and the amount released can vary.

- **There is noise.** The same input can produce different responses. Hear the same song twice and it need not trace the same pattern through your brain each time.
- **There is no global clock.** Nothing ticks all 86 billion cells forward in step. Be careful with what this does *not* say: timing matters a great deal in the brain — the timing of spikes carries information, down to the millisecond. What is missing is a shared metronome, not time itself.
- **The strengths themselves change.** Synaptic strengths can grow and shrink with activity and experience. These changes are part of how the brain learns.

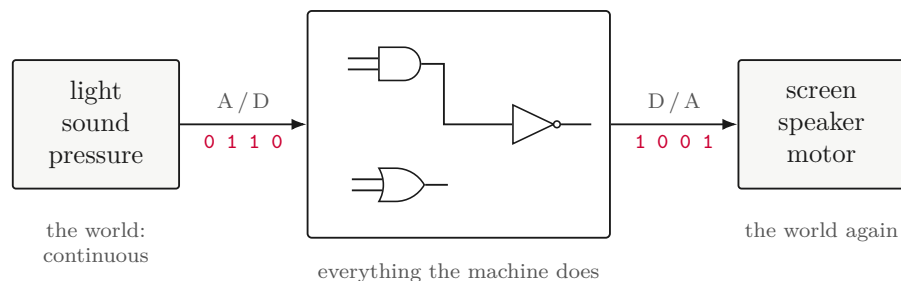
A familiar example of adjustment: step out of a dark room into bright sunlight. At first it is overwhelming; then your visual system adjusts its sensitivity. Several mechanisms contribute, so this is not simply a change in synaptic strengths, or necessarily an example of learning.

These features — continuous voltages, noise, no global clock, and changing strengths — matter when modeling a neuron. Digital circuits also have physical voltages and noise, but they are engineered to give reliable answers in terms of just two symbols.

Act 2. Logic Gates and Certainty

2.1 The machine, from very far away

Seen from far enough away, a computer is a three-stage story:



The world on the left is continuous — light, sound, pressure, voltages from a microphone. At the machine’s boundary all of it is *converted*: measured, and written as strings of 0s and 1s. In the middle, everything the machine does — arithmetic, text, video, this very document — is operations on those 0s and 1s, nothing else. At the far boundary some of the results are converted back to the continuous world: pixels on a screen, sound from a speaker, motion in a motor. Not all of them come back out: a file written to disk or a message sent across the network stays 0s and 1s the whole way. Storage is physical too — a disk encodes each bit as the state of some small piece of solid matter — but what the physical state encodes is a discrete symbol.

Two side remarks belong here. First, “discrete” is a design choice, not a law of nature: there was once a whole branch of computing built on continuous signals — analog computers — and the quantum machines now being developed are different again. The machines this course runs

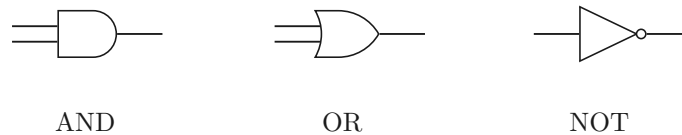
on are the discrete kind. Second, the middle of the picture usually uses a **clock** to coordinate when intermediate results are stored. The logic between those storage points takes time to settle. The brain of Act 1 has no such global clock.

2.2 Two symbols, and nothing in between

Physically, a wire carries a voltage. One range is interpreted as *high*, or **1**, and another as *low*, or **0**. The voltages in between occur during transitions, but do not represent a valid logic value. The circuit must have time to settle before its answer is read. Small voltage changes within either valid range leave the represented bit unchanged; this is part of what makes digital computation reliable.

2.3 Three gates

The operations performed on bits are built from **logic gates**: small devices that, in the examples below, take one or two bits and produce one bit, by a rule small enough to write out in full.



AND answers 1 only when both inputs are 1. **OR** answers 1 when at least one input is 1. **NOT** takes a single input and answers with the other symbol. In class you filled these tables in yourself; here they are complete:

<i>a</i>	<i>b</i>	AND	<i>a</i>	<i>b</i>	OR	<i>a</i>	NOT
0	0	0	0	0	0	0	1
0	1	0	0	1	1	1	0
1	0	0	1	0	1		
1	1	1	1	1	1		

A **truth table** specifies the output for every possible binary input. These four rows completely define the two-input AND function. They do not describe physical details such as how long a gate takes to produce its answer.

Gates also come in wider versions: an AND with three or more inputs answers 1 exactly when *all* of its inputs are 1, and real chips use such gates freely because they save space. Everything in this unit works with the two-input versions, and nothing is lost.

2.4 How many kinds of part does a computer need?

Inside a real processor you would find implementations of several different logical operations — the three above, XOR, and other variants — because dedicated circuits are efficient. But that is engineering convenience. The interesting question is: how many *different kinds* of gate do you actually need, to build all the logic of a computer?

Three are enough: AND, OR and NOT can express any Boolean function. Can you do better? A student in class pointed out that two suffice — AND and NOT — because OR can

be assembled out of them: a OR b answers 0 only when both inputs are 0, which is the same as NOT (NOT a AND NOT b).

And you can do better still. *One* kind of gate is enough: the **NAND**, which is simply AND followed by NOT — it answers with the opposite of what AND answers.

a	b	NAND
0	0	1
0	1	1
1	0	1
1	1	0

Each of the three basic gates can be written using nothing but NAND:

- NOT a = NAND(a, a) — one NAND, both inputs fed from the same wire
- a AND b = NAND(NAND(a, b), NAND(a, b))
- a OR b = NAND(NAND(a, a), NAND(b, b))

One NAND, then two, then three — and no second kind of gate anywhere in the list. NAND gates can therefore build all the Boolean logic a processor needs. Arrangements with feedback can also store bits.

Why NAND rather than AND? Because of what gates are made of. In standard two-input CMOS circuits, a NAND takes four transistors, while an AND is built as a NAND followed by an inverter: six. The inverting gate is the cheap one, so it is a useful building block in chips. The name misleads a little: “NAND” sounds like an AND with a NOT bolted on, but physically it is the AND that is the composite.

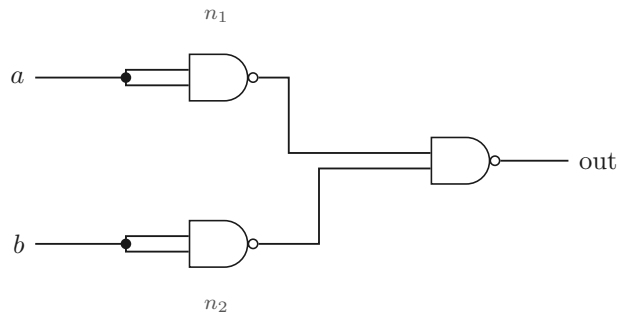
Repeated parts. The brain contains many cells of each type and recurring circuit patterns. A computer also combines repeated small parts into larger modules. NAND shows how far this idea can go: one kind of gate is enough to build every Boolean function.

2.5 Wiring gates together

The output of one gate can feed an input of another, and every computation the machine performs is, at bottom, such a wiring. The way to understand a small circuit is to *trace* it: fix the inputs, and push the values through gate by gate.

Example 2.1 Trace the circuit

Three NANDs, wired as below — the circuit you traced on the meeting’s worksheet. The values on the two internal wires are named n_1 and n_2 .



Take the row $a = 0, b = 0$. The top gate receives a on both inputs, so $n_1 = \text{NAND}(0, 0) = 1$; likewise $n_2 = 1$; and the output is $\text{NAND}(1, 1) = 0$. Now $a = 0, b = 1$: the top gate still sees $(0, 0)$, so $n_1 = 1$; the bottom gate sees $(1, 1)$, so $n_2 = 0$; and the output is $\text{NAND}(1, 0) = 1$. Before computing the row $(1, 0)$, notice that the circuit is *symmetric*: swapping a and b swaps the roles of n_1 and n_2 and cannot change the output. So $(1, 0)$ gives 1 with no further work. The last row, $(1, 1)$, gives $n_1 = n_2 = 0$ and output $\text{NAND}(0, 0) = 1$.

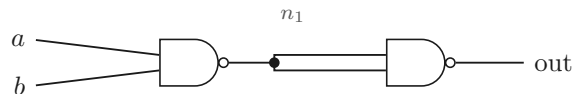
a	b	n_1	n_2	out
0	0	1	1	0
0	1	1	0	1
1	0	0	1	1
1	1	0	0	1

Read the last column: 0, 1, 1, 1. This three-NAND circuit *is* the OR gate — it is the third identity of the previous section, verified row by row.

On paper we evaluated the gates one at a time. In hardware, n_1 and n_2 can settle concurrently because neither depends on the other; the final gate depends on both. Also notice the shortcut for $(1, 0)$: spotting a symmetry can save a calculation. We will use it again in Act 3.

Example 2.2 The same method, for AND

The second identity of §2.4 says that AND is two NANDs: the first takes a and b , and the second takes the first's output on both of its inputs. Drawn as a circuit, with the one internal wire named n_1 :



The method is the one of Example 2.1, and it is the same for every circuit: fix the inputs, push the values through the gates in the order they are wired, and write down each internal wire as you reach it. For $(0, 0)$: $n_1 = \text{NAND}(0, 0) = 1$, and the output is $\text{NAND}(1, 1) = 0$. For $(0, 1)$ and $(1, 0)$: $n_1 = 1$ again, so the output is 0 again. For $(1, 1)$: $n_1 = \text{NAND}(1, 1) = 0$, and the output is $\text{NAND}(0, 0) = 1$.

<i>a</i>	<i>b</i>	<i>n</i> ₁	out
0	0	1	0
0	1	1	0
1	0	1	0
1	1	0	1

The last column is 0,0,0,1: AND. Notice what the second gate is doing: fed the same value on both inputs, a NAND simply inverts it, which is the first identity of §2.4 (NOT $a = \text{NAND}(a, a)$) appearing as a part of a larger circuit. Read this way, the three identities are not three facts to memorize but one method applied three times.

2.6 A gate is exact

In the ideal logic model, a two-input gate's four-row table specifies its entire input-output behavior.

- The same binary inputs give the same answer, every time.
- A gate remembers nothing about any input before this one. It answers, and is done.
- In a synchronous circuit, the clock allows time for answers to settle before they are stored.

Real gates are designed to implement that model reliably within their operating limits. They can fail, but their intended rule does not change with experience. That is the useful contrast with a synapse whose strength changes as the brain learns.

2.7 A gate you can run

A gate is fixed hardware. But an entire computer is built out of gates, programming languages run on the computer — and in a programming language you can write the gate back down as code:

```
class AndGate:
    def __call__(self, a, b):
        return 1 if (a == 1 and b == 1) else 0

gate = AndGate()
for a, b, expected in [(0,0,0), (0,1,0), (1,0,0), (1,1,1)]:
    assert gate(a, b) == expected
```

The loop at the bottom deserves a second look: it is the truth table, row by row. The table is not a description of the class for a reader to consult; it is the requirement the class has to meet, and the loop checks that requirement on every row. A gate class that passes its own truth table is correct for all of its binary inputs. This is one consequence of a gate's exactness: its specification is complete, and small enough to test in full.

Notice also the loop in the construction itself: physical gates make a processor, the processor runs Python, and inside Python we have just rebuilt a gate — as software. Put enough of these software gates together and you could simulate an entire computer, inside the computer.

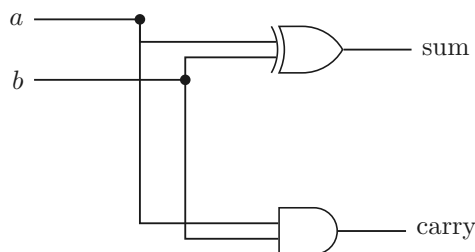
Engineers use circuit simulation to check chip designs before anything is manufactured, although their tools model more hardware detail than these Python classes.

Toward the assignment. Task 1 of the unit assignment asks for exactly what this section did once: the four gate classes — AND, OR, NOT, NAND — each tested against its own truth table, in your own notebook.

Act 3. Computing a Neuron, Approximately

3.1 Gates can count

Wired together, gates do arithmetic. Two of them add one bit to one bit:



The XOR gate (1 when *exactly one* input is 1) produces the sum bit, and an AND produces the carry — check it against $1 + 1$, which is 10 in binary: sum 0, carry 1. This circuit is a *half adder*. To add longer binary numbers, each position must also accept the carry from the preceding position. A *full adder* does that; it can be built from two half adders and an OR gate. Chain full adders and you can add 64-bit numbers. Multiplication is shifting and adding; comparison, subtraction, and the rest of arithmetic follow. Every number the machine ever adds, it adds *with gates* — there is no other machinery underneath.

Arithmetic was a major purpose of early computers. A pocket calculator is a familiar use of the same idea: gates carrying out arithmetic. (A game to play with one: type a large positive number and press the square-root key repeatedly. The display marches toward 1 — fast at first, then very slowly. Working out *why* it slows down is a good exercise for later in the course.)

3.2 Turn the question around

So: arithmetic is built out of gates.

Could a **neuron** be built out of **arithmetic**?

The computer already performs additions, multiplications, and comparisons. We can use those operations to write a model of a neuron as a program.

3.3 The cell, written as arithmetic

Return to the four-step description of Act 1, and translate each step — in order — into a piece of arithmetic. This is a *modeling* step: deciding what each biological feature becomes, as a number or an operation.

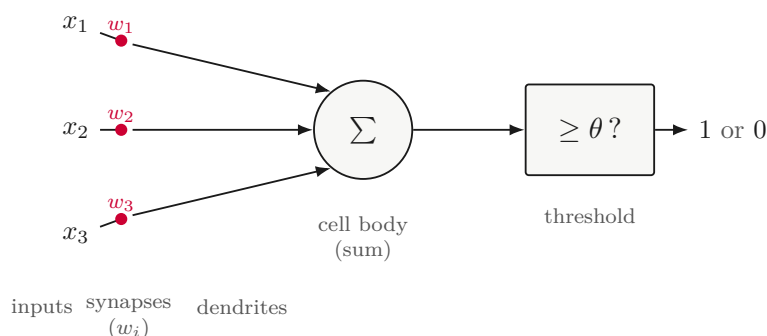
signals cross the synapses	inputs $x_1, \dots, x_n$
each synapse has a strength	a weight w_i for each input
the cell body adds what arrives	$w_1x_1 + w_2x_2 + \dots + w_nx_n$
it fires past a threshold	output 1 if the sum is $\geq \theta$, else 0

In prose: the incoming signals become numbers $x_1, \dots, x_n$ — real numbers, since the signals they stand for are continuous. Each synapse’s strength becomes a number too, the **weight** w_i : a large weight is a synapse paying close attention to its input, a weight near zero is one ignoring it. For a positive input, a negative weight pushes the cell away from firing. The cell body’s gathering becomes multiplication and addition — each input counted in proportion to its weight — and the firing decision becomes a comparison against the **threshold**, which by long tradition gets a Greek letter: θ . (Every course acquires its Greek letters eventually; this is our first.)

Writing $w_1x_1 + w_2x_2 + \dots + w_nx_n$ in full gets old quickly. A student in class supplied the standard shorthand, which says exactly the same thing:

$$\sum_{i=1}^n w_i x_i \quad \text{“the sum, over } i \text{ from 1 to } n, \text{ of } w_i x_i \text{.”}$$

The whole object — inputs, weights, sum, threshold — drawn as one device:



Three inputs are drawn; there could be any number. Notice that the picture is the neuron of Act 1 with every part replaced by its arithmetic counterpart — and that the weights and the threshold are the parameters we choose.

Example 3.1 Does it fire?

Fix a unit with three inputs, weights $w = (2, -1, 2)$, and threshold $\theta = 3$ — the unit from the meeting’s worksheet. For each input, form the weighted sum and compare it with θ .

For $x = (1, 0, 1)$: the sum is $2 \cdot 1 + (-1) \cdot 0 + 2 \cdot 1 = 4$, and $4 \geq 3$, so the unit fires: output 1.

For $x = (1, 1, 1)$: the sum is $2 - 1 + 2 = 3$ — *exactly* the threshold. Which way does it go? Intuition cannot settle this; only the definition can, and our definition says the unit fires when the sum is *greater than or equal to* θ . Output 1. The row is a small lesson in reading definitions literally: a boundary case is decided by the written rule, not by feel. (And the rule has consequences: raise the threshold to 3.5 and this row, alone among the four, changes its answer.)

For $x = (0, 1, 1)$: the sum is $0 - 1 + 2 = 1 < 3$. Silent: output 0.

For $x = (1, 1, 0)$: no arithmetic needed. The first and third weights are equal, so swapping x_1 and x_3 cannot change the sum — this is the same symmetry shortcut as in Example 2.1. The sum is again 1, and the output 0.

x_1	x_2	x_3	sum	out
1	0	1	4	1
1	1	1	3	1
0	1	1	1	0
1	1	0	1	0

One more question: *which function is this?* For binary inputs, it is AND on x_1 and x_3 , regardless of x_2 . When both are 1, the sum is either 4 or 3, so the unit fires. If either is 0, the sum is at most 2. Check the remaining four rows to complete the truth table.

3.4 McCulloch and Pitts, 1943

In 1943 Warren McCulloch, a neurophysiologist, and Walter Pitts, a logician, proposed an early threshold model with binary signals and excitatory and inhibitory inputs. Our unit with adjustable numerical weights is a related, more flexible model. Their collaboration is an early example of the connections between neuroscience, mathematics, logic, and computing.

Their work connected neuron models with logic. In the notation we use here, the main ideas are:

- Weights and threshold can be *chosen* so that one unit reproduces a named gate exactly. There is a setting for AND, a setting for OR, a setting for NOT, a setting for NAND. (Try to find some of these settings yourself — for AND with two inputs, what weights and threshold work?)
- Wired into networks, such units can compute *any* logical function — so a complete computing machine could, in principle, be assembled from them.

The connection is useful in both directions: gates perform the arithmetic of a neuron model, and a neuron model with suitable parameters can reproduce a gate’s logic.

W. S. McCulloch and W. Pitts, “A logical calculus of the ideas immanent in nervous activity,” *Bulletin of Mathematical Biophysics* 5 (1943), 115–133.

Gates, twice in one meeting. Gates have now appeared in two entirely different roles. As the *thing to imitate*: AND, OR and NOT are functions a unit can be asked to reproduce. And as the *material*: the additions and the comparison the unit performs are themselves carried out by gates.

3.5 Approximately

What the model *keeps* from the cell: weighted inputs, accumulation, a threshold, an all-or-none response. What it *leaves out*: everything else in Act 1. There is no timing and no spike rate — the unit gives one answer, once. There is no chemistry, no noise, no gap to cross. These are deliberate simplifications. The model allows real-valued inputs and weights; a computer represents them with finite precision.

An MP3 is a useful analogy: a sampled, compressed recording leaves out some information, yet it can preserve much of what we enjoy in the music. Similarly, a simplified neuron can retain useful structure without reproducing every biological detail.

(The class also raised a broader question: could continuous, or even quantum, details of the biology matter for understanding the mind and consciousness? Our threshold model does not answer that question. Here we are studying what a simplified model can do.)

One omission matters more than all the others, and it is the one to carry out of this meeting. In the model used here, the weights are numbers *we* choose, and they stay where we put them. In a living brain, synaptic strengths can change with experience. Every weight in Example 3.1 was picked by hand. The unit question — *written, or taught?* — is asking whether that gap can be closed.

3.6 Coda: on making mistakes

One more thread from the meeting: give yourself opportunities to learn from mistakes. Trying something, checking it, and adjusting helps you discover what you do and do not understand. There is a useful parallel with artificial neural networks: training repeatedly scores their outputs and adjusts their weights. People learn in more ways than this, but the practical advice holds: do the work, inspect your errors, and try again.

The class also made a rough energy comparison. A human brain runs on about 20 watts, or about 175 kilowatt-hours over a year. For scale, a hypothetical training run using a million kilowatt-hours would use as much energy as roughly 5,700 years at that power. That compares energy, not years of learning: a brain uses energy for much more than education, and model training costs vary widely. The advice does not depend on the estimate. Use whatever tools you like, but arrange your work so that you make your own attempts and understand your own mistakes.

After this meeting.

- Set up your environment: **Python 3 and Jupyter with NumPy** — Anaconda locally, or Colab in a browser.
- Assignment Task 1, *Gates by hand*: the four gate classes (AND, OR, NOT, NAND), each tested against the truth table you filled in during the meeting.
- Review Quiz 1 is on Canvas.